

```
# Backend Core Infrastructure Reference
# Production E-Commerce Platform – Django + DRF + Celery

> **Purpose of this document**
> This file is the complete reference for the core infras
> Django backend. Any AI model or developer onboarding to
> this entire document before writing a single line of co
> convention, and constraint described here is already im
> production use. Do not invent alternatives. Do not gues
> file if you need something not covered here.
```

Table of Contents

1. Project Overview
2. Technology Stack
3. Application Registry
4. Six-Layer Architecture
5. API Response Envelope – The Contract
6. Exception Hierarchy
7. Global Exception Handler
8. BaseAPIView – All Views Inherit This
9. APIResponseMixin
10. Error Codes Registry
11. Two-Level Cache System
12. Pagination System
13. Serializer Fields and Mixins
14. Permission Classes
15. Middleware
16. Authentication and Token Strategy
17. Logging System
18. Settings Reference
19. Critical Rules – Never Violate
20. How to Add a New Endpoint – Checklist
21. Appendix A – Response Examples by Scenario
22. Appendix B – Import Reference

1. Project Overview

This is a production-scale e-commerce backend built for t
It follows Daraz-style patterns: verified purchases, COD
handling, local courier integrations (PostEx, TCS, Leopard
(Return to Origin) tracking.

Total Apps: 19 (3 core/infrastructure + 16 domain)
Total Domain Models: 29+

The backend is a REST API consumed by a React frontend. A
via Postman. The architecture enforces strict consistency
returns the identical envelope shape regardless of succes

2. Technology Stack

Layer	Technology
Web Framework	Django 4.x
API Framework	Django REST Framework (D
Async Task Queue	Celery
Message Broker / Cache	Redis (port 6380 – NOT d
Database	PostgreSQL
Authentication	SimpleJWT (JWT tokens)
API Documentation	drf-spectacular (OpenAPI
Query Layer	GraphQL via graphene-dja
Static Files	WhiteNoise
Error Monitoring	Sentry SDK
Environment Config	django-environ
Dev Tooling	django-debug-toolbar, dj

3. Application Registry

Infrastructure Apps (no domain logic)

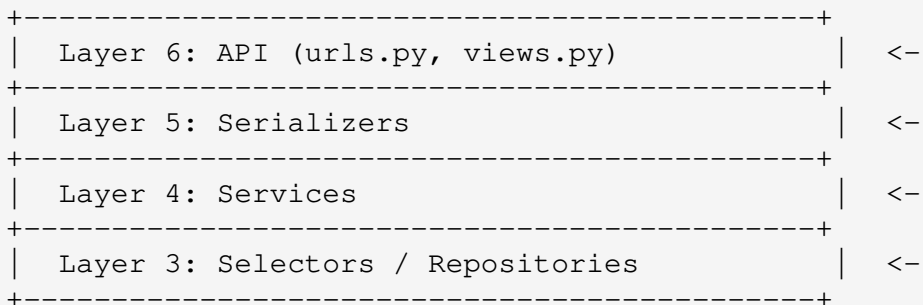
- apps.core - Base views, exceptions, cache, pag
 middleware, error codes, logging f
- apps.common - Shared choices (Role, etc.), share

Domain Apps

- apps.accounts - Auth, JWT, profiles, addresses
- apps.products - Catalog, variants, attributes,
- apps.cart - Cart and cart items
- apps.coupons - Coupon engine
- apps.orders - Order creation, state machine,
- apps.payments - Payment gateway integrations
- apps.reviews - Verified-purchase reviews, voti
- apps.contact - Customer support messages and r
- apps.wishlist - Wishlist items
- apps.tax - Tax calculation
- apps.returns - Return requests
- apps.recomendations - Interaction logging, personaliz
- apps.notifications - Push / email notifications
- apps.logistics - Shipments, courier webhooks, CO
- apps.analytics - Daily snapshots, courier perfor

4. Six-Layer Architecture

Every domain app follows this strict layer separation.
Imports must only flow downward. Upper layers import lower



	Layer 2: Models		<-
+	-----	+	
	Layer 1: Core / Common		<-
+	-----	+	

Rules:

- Views call services, never query models directly
- Services raise DomainError or InfrastructureError,
- Serializers validate input, they do not contain bus
- Models contain field definitions and simple model-l
- The global exception handler converts all exception

5. API Response Envelope – The Contract

CRITICAL: This envelope shape is returned by EVERY endpoint without exception. The frontend relies on this. Never bre

```

{
  "success": true,
  "message": "Human readable summary",
  "data":    {},
  "errors":  null,
  "meta": {
    "request_id": "uuid-string"
  }
}

```

Field Rules

Field	Success (2xx)	Er
-----	-----	----
success	true	fa
message	Confirmation string	Sa
data	Payload object/array	nu
errors	null (always)	Er
meta	Always present, contains request_id	Al

Success Response – Full Example

```

{
  "success": true,
  "message": "Account created successfully. Please
"data": {
    "email": "user@example.com"
  },
  "errors": null,
  "meta": {
    "request_id": "a1b2c3d4-e5f6-7890-abcd-ef1234
  }
}

```

Error Response – Full Example

```

{
  "success": false,
  "message": "Registration failed.",
  "data": null,
  "errors": {
    "code": "validation_error",
    "fields": {
      "email": {
        "message": "Enter a valid email address",
        "code": "invalid_email_format"
      },
      "password": {
        "message": "Password must be at least 8 characters",
        "code": "min_length"
      }
    },
    "non_fields": null
  },
  "meta": {
    "request_id": "a1b2c3d4-e5f6-7890-abcd-ef1234
  }
}

```

The errors Object – Complete Structure

```

"errors": {

```

```

"code": "top_level_error_code",
"fields": {
    "<field_name>": {
        "message": "Human readable error for this",
        "code": "machine_readable_code"
    }
},
"non_fields": {
    "category": "validation | domain | system | u",
    "message": "Human readable error",
    "code": "machine_readable_code",
    "extra": {} | null
}
}

```

fields – When Populated

- Serializer field-level validation failures only
- Each key = field name that failed
- Only the FIRST error per field is surfaced (DRF ret
- Multiple field errors returned simultaneously across

non_fields – Four Categories

Category	When Used
validation	DRF non_field_errors and DRF-native exceptions already recognizes (e.g. malformed serializations) and rejects before reaching a view's business logic.
domain	Any BaseAppError subclass whose category is now includes not just generic business-as-usual errors (e.g. DomainError), but also NotFoundError, AuthenticationRequiredError (401), and since all three are DomainError subclasses, this is a change from the previous version where not-found/unauthorized/forbidden responses were returned with category="validation". They are categorized as domain errors because that is what the exception class they are actually is.
system	Infrastructure failure (raised as InfraError)

unexpected | Unhandled exception – always triggers m

IMPORTANT FOR FRONTEND: if any frontend logic current non_fields.category == "validation" to detect a 404/ must be updated to check non_fields.category == "dom better, branch on the top-level errors.code (not_foun permission_error) which has not changed and remains t to switch on. errors.code is unaffected by this chang changed for these three specific response types.

HTTP Status to errors.code Mapping

HTTP Status	errors.code
-----	-----
400	validation_error
401	authentication_error
403	permission_error
404	not_found
405	method_not_allowed
409	conflict_error
422	validation_error
429	rate_limit_exceeded
500	server_error
503	server_error

NOTE: 409 maps to conflict_error, NOT validation_error
Rationale: validation_error means malformed data.
409 means data was valid but current state refuses th
The frontend branches on the top-level code – confla

NOTE: 422 was added to this mapping to support Domain
ALLOWED_STATUS_CODES (see Section 6). It maps to vali
as 400 – both represent "the request itself was inval
more specific status some clients expect for semantic
well-formed input.

HTTP Status Default Messages (safe, shown to users)

HTTP Status	Message
-----	-----

```

400      | "Invalid request data."
401      | "Authentication required."
403      | "You do not have permission to perform
404      | "The requested resource was not found."
405      | "Method not allowed."
409      | "This action conflicts with the current
422      | "Invalid request data."
429      | "Too many requests. Please slow down."
500      | "An unexpected error occurred. Please t
503      | "The service is temporarily unavailable

```

Paginated Response – Meta Structure

When an endpoint returns a paginated list, the meta field

```

{
  "success": true,
  "message": "...",
  "data": [...],
  "errors": null,
  "meta": {
    "request_id": "...",
    "page": 1,
    "page_size": 20,
    "total": 540,
    "total_pages": 27,
    "showing_from": 1,
    "showing_to": 20,
    "has_next": true,
    "has_previous": false,
    "source": "l2_redis",
    "sort": "newest",
    "elapsed_ms": 12.4
  }
}

```

source, sort, and elapsed_ms are optional.
 null values are excluded from meta entirely – do not

6. Exception Hierarchy

File: apps/core/exceptions.py

CRITICAL RULE: This file has ZERO imports from Django HTTP and Django apps.core.api. It is safe to import from any layer including Django and management commands. This file is the single source of truth for what every exception IS – its status code, its category, its machine-readable code, and whether it notifies monitoring. No file outside this one needs to know the difference between exception types to handle them correctly; every exception describes

```
BaseAppError(Exception)
├── DomainError                → Business rule violation
│   ├── NotFoundError         → 404, category="domain"
│   ├── AuthenticationRequiredError → 401, category="domain"
│   └── PermissionDeniedError  → 403, category="domain"
└── InfrastructureError        → Infrastructure failure
```

BaseAppError

Never raise directly. Exists as the shared root every other exception inherits from, and as the type custom_exception_handler checks for a single isinstance(exc, BaseAppError) branch – this is why all exception subclasses work end-to-end without any change to

```
BaseAppError(
    message: str,                # human-readable message
    *,
    code: str | None = None,      # machine-readable code
                                   # to the status code
    client_extra: dict | None = None, # forwarded client data
    internal: dict | None = None,     # NEVER for client
    notify: bool | None = None,       # overrides default behavior
)

```

Class-level attributes every subclass declares (rather than instance logic in `__init__`):

```

.status_code      = 500          # overridden per subclass
.category         = "unexpected" # overridden per subclass
                                # handler for logging
.default_code     = "app_error"  # overridden per subclass
.default_notify   = False        # overridden per subclass

```

Method every subclass inherits and should not need to override

```

.to_envelope() -> dict
    Returns {"category": ..., "message": ..., "code": ...}
    This is the ONLY function in the entire codebase
    a BaseAppError is described to a client. It deliberately
    internal – that exclusion happening here, once, is what
    is what makes "internal never reaches a response"
    guarantee instead of something every call site has to
    Both custom_exception_handler and BaseAPIView.app_error
    call this same method on the same instance, which
    guarantees "raise this exception" and "manually build
    from this exception" always produce identical output

```

DomainError

Use when the domain says NO to a valid request.

Not a malformed request – the data was fine but business

```

DomainError(
    message: str,
    *,
    code: str = "domain_error",
    status_code: int = 400,          # MUST be one of ALLOWED_STATUS_CODES
                                    # – raises ValueError if not
                                    # construction time
    client_extra: dict | None = None,
    internal:      dict | None = None,
)
ALLOWED_STATUS_CODES = {400, 401, 403, 404, 409, 422}

```

NOTE: This set was widened from the previous {400, 404} to include
the named subclasses below (NotFoundError=404, AuthnError=409)

PermissionDeniedError=403) plus 422 for semantic-validation without removing the fail-fast validation guard itself. status_code still raises ValueError immediately, namely:

```
.category          = "domain"
.default_code       = "domain_error"
.default_notify     = False      # DomainError never alerts
                                # it represents expected but
```

status_code is set as an INSTANCE attribute in DomainError (not a class attribute), because individual DomainError instances vary in status – 400 for one call site, 409 for another. from InfrastructureError below, where status_code is a class attribute because every instance is always 503.

When to use which status:

- 400 – Generic domain rejection: "Cart is empty", "Credentials are invalid"
- 401 – No valid credentials on the request (or raise AuthenticationError)
- 403 – Valid credentials, insufficient access (or raise AccessError)
- 404 – Resource does not exist (or raise NotFoundError)
- 409 – State conflict: "Order already cancelled", "Email already taken"
- 422 – Semantically invalid but well-formed input, where validation fails

Examples:

```
# 400 – generic rejection
raise DomainError(
    "Your cart is empty. Add items before checking out.",
    code="cart_empty",
)

# 409 – state conflict
raise DomainError(
    "This order has already been cancelled.",
    code="order_already_cancelled",
    status_code=409,
)

# 409 with client_extra (forwarded to response) and i
raise DomainError(
```

```

        "Coupon has already been used.",
        code="coupon_already_used",
        status_code=409,
        client_extra={"applied_on": "2024-01-15"},
        internal={"coupon_id": 99, "used_by_user_id": 12}
    )

```

NotFoundError, AuthenticationRequiredError, PermissionDeniedError

Named DomainError subclasses, added to replace the previous BaseAPIView.not_found_response() / unauthorized_response() hand-formatted a response with category="validation" using formatting rather than a real exception instance. These three first-class exception types, each hardcoding its own status attribute (no per-instance variation needed, same reasoning as

```

NotFoundError(
    message: str = "The requested resource was not found.",
    *,
    code: str = "not_found",
    client_extra: dict | None = None,
    internal: dict | None = None,
)
.status_code = 404, .category = "domain"

```

```

AuthenticationRequiredError(
    message: str = "Authentication required.",
    *,
    code: str = "authentication_error",
    client_extra: dict | None = None,
    internal: dict | None = None,
)
.status_code = 401, .category = "domain"

```

```

PermissionDeniedError(
    message: str = "You do not have permission to perform this action.",
    *,
    code: str = "permission_error",
    client_extra: dict | None = None,
    internal: dict | None = None,
)
.status_code = 403, .category = "domain"

```

```
)
    .status_code = 403, .category = "domain"
```

CONTRACT CHANGE FROM PREVIOUS VERSION: responses produced `BaseAPIView.not_found_response()` / `unauthorized_response()` now have `non_fields.category = "domain"`, not `"validation"` (`not_found` / `authentication_error` / `permission_error`) is Section 5's `"non_fields – Four Categories"` note above for implication.

These can also be raised directly from a service layer, e.g. `DomainError`:

```
raise NotFoundError(f"Order {order_id} was not found.")
raise AuthenticationRequiredError()
raise PermissionDeniedError("You cannot cancel another order")
```

InfrastructureError

Use when an external system or internal subsystem is temporarily unavailable. Always returns 503. Not configurable per instance.

IMPORTANT: Named `InfrastructureError` NOT `SystemError` to avoid shadowing Python's built-in `SystemError`, which the interpreter and CPython use for internal VM-level failures. Shadowing it causes silent failures in third-party libraries that catch `SystemError`.

```
class InfrastructureError(BaseAPIException):
    message: str,
    *,
    code: str = "infrastructure_error",
    notify: bool = True,          # True = alert Sentry/PagerDuty
                                # False = WARNING log only
    client_extra: dict | None = None,
    internal: dict | None = None,
)
    .status_code      = 503  (class attribute, same for every subclass)
    .category         = "system"
    .default_code     = "infrastructure_error"
    .default_notify   = True  # infrastructure failures are always notified
```

```
# pass notify=False at the
# specific instance represe
# handled, expected outage
```

Examples:

```
# Unexpected failure – alert on-call
raise InfrastructureError(
    "Payment gateway is temporarily unavailable. Plea
    code="payment_gateway_timeout",
    client_extra={"retry_after": 30},
    internal={"gateway": "stripe", "http_status": 504
)

# Expected, handled outage – fallback in place, no pa
raise InfrastructureError(
    "Recommendations unavailable. Showing defaults.",
    code="recommendation_service_down",
    notify=False,
    internal={"service": "rec-engine", "reason": "cir
)
```

Adding a New Exception Type

Because every exception describes itself via class attrib to_envelope(), adding a new type requires editing ONLY th (apps/core/exceptions.py). No change is needed in apps/core/api/exceptions.py or apps/core/api/views.py for to work end-to-end through the global handler and any vie raises it directly.

```
class RateLimitError(BaseAppError):
    status_code      = 429
    category         = "system"
    default_code     = "rate_limit_exceeded"
    default_notify   = False    # expected, not a defec
```

That is the entire change required.

7. Global Exception Handler

File: apps/core/api/exceptions.py

Registered in settings: "EXCEPTION_HANDLER": "apps.core.a

THE SINGLE MOST IMPORTANT ARCHITECTURAL INVARIANT:

All logging lives in `custom_exception_handler` and ONLY th

Builder functions are pure – input in, dict out, zero sid

This guarantees exactly ONE log entry and ONE Sentry even

SECURITY INVARIANT:

`exc.internal` is NEVER passed to any builder function and

any response. It is only written to the logger inside cus

This invariant is now enforced structurally by `BaseAppErr`

itself (see Section 6) rather than repeated by hand in ev

this handler.

PUBLIC INTERFACE OF THIS MODULE:

Only three names are meant to be imported by other module

<code>custom_exception_handler</code>	– DRF <code>EXCEPTION_HANDL</code>
<code>register_monitor</code>	– monitoring backend
<code>build_envelope_from_exception</code>	– shared envelope bui
	<code>BaseAppError</code> , used
	handler and <code>BaseAP</code>
	cannot produce dif
	for the same excep

Every other name in this file is underscore-prefixed and

outside this file should import an underscore-prefixed na

Decision Tree (evaluated in order)

```
Exception raised
|
|— isinstance(exc, BaseAppError)?
|   [single branch – covers DomainError, Infrastr
|   NotFoundError, AuthenticationRequiredError,
|   and any future subclass, with no per-type br
```

```

        status                = exc.status_code
        non_fields.category = exc.category (read from
                                           not hardcoded per exce
        if exc.notify=True → _notify_monitors(exc) +
        if exc.notify=False → WARNING log, no traceback
        exc.internal        = logged (at ERROR if not
                               notify=False) – never
        response body       = build_envelope_from_exc
                               internally calls exc.t

└─ DRF-recognised exception? (exception_handler returns No
    status                = original DRF status code pr
    non_fields.category = "validation"
    if status >= 500 → _notify_monitors + ERROR 1
    if status < 500 → WARNING log, no traceback
    response body     = _format_drf_errors(response

└─ Unhandled exception (exception_handler returns No
    status                = 500
    non_fields.category = "unexpected"
    always                → _notify_monitors + ERROR lo
    DEBUG=True            → raw exc message in response
    DEBUG=False           → safe generic message only (

```

NOTE: this handler previously had TWO separate branches: `isinstance(exc, DomainError)`, one for `isinstance(exc, BaseAppError)` – each with hand-written logging logic. They have been replaced by the single `BaseAppError` branch above, driven generically by `exc.category` rather than by which class `exc` happens to be. The shape for existing `DomainError` and `InfrastructureError` is unchanged; only the code path producing that output changed.

Monitor Registry

```

from apps.core.api.exceptions import register_monitor

# Register Sentry or any monitor – called once at startup
register_monitor(lambda exc: capture_exception(exc))

```

Monitors ARE called for:

- Any BaseAppError instance where exc.notify is True (InfrastructureError defaults to notify=True; DRF subclasses default to notify=False, but any instance can override this via the notify kwarg)
- DRF 5xx responses
- Unhandled exceptions

Monitors are NOT called for:

- Any BaseAppError instance where exc.notify is False (DomainError and subclasses by default – expect subclasses to override)
- DRF 4xx (client errors, not our defects)

Key Functions

`build_envelope_from_exception(exc: BaseAppError) -> dict`
 Returns {"code": ..., "fields": None, "non_fields_errors": ...}
 This is the single function that turns ANY BaseAppError into a full errors envelope. Called by custom_exception_handler every BaseAppError branch, and by BaseAPIView.app_exception_handler for the same purpose – both call sites are guaranteed identical output because they call this same function on the same instance.

`_format_drf_errors(data, status_code) -> dict` [private]
 Normalizes DRF-native error data into the standard format. Only used for exceptions DRF's own exception_handler recognizes and formatted, or for raw string/dict data if the helper wants formatted the same way. BaseAppError subclasses reach this function.

Handles three DRF shapes:

```
dict    -> {"email": [ErrorDetail], "non_field_errors": ...}
list    -> [ErrorDetail("Authentication credentials not found")]
str      -> "Not found."
```

`_status_to_error_code(status_code) -> str` [private]
 Maps HTTP status to top-level errors.code string. The default fallback source of truth for the coarse top-level error code – the richer per-exception detail (category, specific code) always comes from exc.to_envelope(). 409 maps to conflict_error, not 400.

422 maps to validation_error, same as 400.

```
_status_to_message(status_code) -> str    [private]
    Maps HTTP status to safe default human message.
```

```
_non_fields_validation(detail) -> dict    [private]
    Builds a "validation" non_fields block from a single
    value (ErrorDetail or plain string). Used only for
    exceptions, not for BaseAppError instances.
```

```
_non_fields_unexpected(exc, *, status_code) -> dict
    Builds unexpected non_fields block. DEBUG shows r
    Production always shows safe generic message.
```

REMOVED FROM THIS VERSION: `_non_fields_domain` and `_non_fields_infrastructure` no longer exist as separate methods. Their responsibility is now covered generically by `BaseAppError.to_envelope()` (Section 6) plus `build_envelope_from_exception` above, since every `BaseAppError` subclass – regardless of the exception it wraps – describes itself the same way.

8. BaseAPIView – All Views Inherit This

File: `apps/core/api/views.py`

```
from apps.core.api.views import BaseAPIView

class MyView(BaseAPIView):
    ...
```

RULE: Every view in the project inherits from `BaseAPIView`.
Never use `APIView` or `GenericAPIView` directly.

`BaseAPIView` inherits from: `APIResponseMixin`, `GenericAPIView`.
Default: `permission_classes = [AllowAny]` – override per subclass.

This file imports ONLY the public interface of `apps.core`.
(`build_envelope_from_exception`, plus `_format_drf_errors` and `_format_drf_errors`).

specifically for raw DRF/string error input in `error_response` that method's own note below for why). No underscore-prefixed `apps.core.api.exceptions` is imported here.

Response Helper Decision Guide

Situation	Helper to Use
Successful GET, list, or action	<code>self.successful</code>
Resource just created (POST)	<code>self.created</code>
Serializer field errors	<code>self.error_response</code>
Any <code>BaseAppError</code> caught directly in view	<code>self.app_error_response</code>
Resource does not exist	<code>self.not_found</code>
No auth credentials	<code>self.unauthenticated</code>
Auth present but access denied	<code>self.forbidden</code>

PREFERRED PATTERN: Raise `DomainError` / `InfrastructureError` subclasses in the service layer and let `custom_exception_handler` handle it automatically. Only use `app_error_response` if you directly construct or catch a `BaseAppError` without going through `handle_exception` in between (rare).

BREAKING CHANGE FROM PREVIOUS VERSION

`domain_error_response(exc: DomainError) -> Response`

has been RENAMED to:

`app_error_response(exc: BaseAppError) -> Response`

The new name reflects that it now accepts ANY `BaseAppError` and not only `DomainError` – it works identically for `InfrastructureError`, `NotFoundError`, `AuthenticationRequiredError`, `PermissionDeniedError`, and any future subclass. Any existing call site using `domain_error_response` should be updated to `app_error_response`; the underlying behavior and the same envelope the global handler would produce for the same exception is preserved, and is now a guarantee rather than a documented behavior because both paths call `build_envelope_from_exception(exc)`.

Full Method Signatures

— Success —————

```
def success_response(  
    self,  
    *,  
    data: Any = None,  
    message: str = "Request successful",  
    status_code: int = 200,  
    meta: dict | None = None,  
) -> Response
```

```
def created_response(  
    self,  
    *,  
    data: Any = None,  
    message: str = "Resource created successfully",  
    meta: dict | None = None,  
) -> Response
```

— Error —————

```
def error_response(  
    self,  
    *,  
    message: str = "Request failed",  
    errors: Any = None, # serializer.errors,  
    status_code: int = 400,  
) -> Response  
# Internally calls _format_drf_errors(errors, status_  
# Use this ONLY for raw DRF-shaped data (serializer.e  
# strings, raw dicts) – never for a BaseAppError inst  
# is no exception object here to ask for its own enve  
# exactly the case _format_drf_errors exists for.  
# Do NOT use for domain errors – use app_error_respon  
# so the frontend receives the exception's real categ  
# "domain") rather than "validation".
```

```
def app_error_response(  
    self,
```

```

        *,
        exc: BaseAppError,      # any subclass - DomainErr
                                # NotFoundError, Authenti
                                # PermissionDeniedError,
    ) -> Response
    # Delegates entirely to build_envelope_from_exception
    # function custom_exception_handler calls for an unca
    # the same type. Produces byte-identical output to ra
    # letting the global handler process it, for every su
    # DomainError as in the previous version.

def not_found_response(
    self,
    *,
    message: str = "The requested resource was not fo
) -> Response
# Returns 404. Builds NotFoundError(message) internal
# through app_error_response(). errors.code="not_foun
# non_fields.category="domain" (CHANGED from "validat
# Section 5 and Section 6's NotFoundError note).

def unauthorized_response(
    self,
    *,
    message: str = "Authentication required.",
) -> Response
# Returns 401. Builds AuthenticationRequiredError(mes
# errors.code="authentication_error" (unchanged).
# non_fields.category="domain" (CHANGED from "validat

def forbidden_response(
    self,
    *,
    message: str = "You do not have permission to per
) -> Response
# Returns 403. Builds PermissionDeniedError(message)
# errors.code="permission_error" (unchanged).
# non_fields.category="domain" (CHANGED from "validat

### transform_payload() - Request ID Injection

```

BaseAPIView overrides APIResponseMixin.transform_payload(request_id) into every response meta automatically.

```
def transform_payload(self, payload: dict) -> dict:
    request_id = getattr(self.request, "id", None)
    if request_id:
        if payload.get("meta") is None:
            payload["meta"] = {}
        payload["meta"]["request_id"] = request_id
    return payload
```

request.id is set by RequestIDMiddleware.

If middleware is absent (e.g. unit tests), this is a

NOTE: this method is unchanged from the previous vers

9. APIResponseMixin

File: apps/core/api/mixins.py

BaseAPIView inherits from this. You do not use this direc

NOTE: This file is unchanged in behavior from the previous was already fully decoupled from apps.core.exceptions in and remains so. Only its docstrings and type hints were m the exception-handling refactor; no logic changed.

```
class APIResponseMixin:
    MESSAGE_KEY: str = "message"
    ERRORS_KEY: str = "errors"
    DATA_KEY: str = "data"
    SUCCESS_KEY: str = "success"
    META_KEY: str = "meta"

    def build_response(
        self,
        *,
```

```

        data:          Any = None,
        message:       str | None = None,
        errors:        Any = None,
        status_code:   int = 200,
        meta:          dict | None = None,
        headers:       dict | None = None,
    ) -> Response

```

```

def transform_payload(self, payload: dict) -> dict:
    return payload    # no-op in mixin, overridden in

```

The `build_response()` method:

1. Assembles the 5-key envelope (success, message, data, errors, status_code)
2. Calls `self.transform_payload(payload)` – injects response
3. Creates `Response(payload, status=status_code)`
4. Applies any headers dict to the response object

10. Error Codes Registry

File: `apps/core/error_codes.py`

```

from apps.core.error_codes import ErrorCode

```

NOTE: This file is UNCHANGED by the exception-handling refactor. It is a flat registry of string constants with no dependency on `apps.core.exceptions` or `apps.core.api.exceptions` in either direction. The refactor did not touch it, and no update to this section of the note is required. It remains accurate as originally documented.

All machine-readable error codes live here.

No string literals at raise sites – always use `ErrorCode`. These are a PUBLIC API. The React frontend switches on them. Changing a value string is a BREAKING CHANGE.

```

# — Authentication —————
ErrorCode.INVALID_CREDENTIALS      = "invalid_credentials"
ErrorCode.EMAIL_NOT_VERIFIED       = "email_not_verified"
ErrorCode.ACCOUNT_DISABLED         = "account_disabled"

```

ErrorCode.ACCOUNT_INACTIVE	= "account_ina
ErrorCode.SESSIO	= "session_exp
ErrorCode.TOKEN_INVALID	= "token_inval
ErrorCode.TOKEN_EXPIRED	= "token_expir

— OTP —

ErrorCode.OTP_RESEND_COOLDOWN	= "otp_resend_
ErrorCode.OTP_NOT_FOUND	= "otp_not_fou
ErrorCode.OTP_EXPIRED	= "otp_expired
ErrorCode.OTP_INVALID	= "otp_invalid
ErrorCode.OTP_ATTEMPTS_EXCEEDED	= "otp_attempt

— Verified Session —

ErrorCode.VERIFICATION_SESSION_INVALID	= "verificatio
ErrorCode.VERIFICATION_SESSION_EXPIRED	= "verificatio

— Social Auth —

ErrorCode.UNSUPPORTED_AUTH_PROVIDER	= "unsupported
ErrorCode.INVALID_SOCIAL_TOKEN	= "invalid_soc
ErrorCode.AUTH_PROVIDER_UNREACHABLE	= "auth_provid
ErrorCode.SOCIAL_EMAIL_NOT_VERIFIED	= "social_email
ErrorCode.SOCIAL_EMAIL_MISSING	= "social_email

NOTE: ACCOUNT_INACTIVE appears in both Auth and Soc

— Registration / Email —

ErrorCode.EMAIL_ALREADY_EXISTS	= "email_alrea
ErrorCode.EMAIL_INVALID_FORMAT	= "email_inval

— User / Profile —

ErrorCode.INVALID_FULL_NAME	= "invalid_ful
ErrorCode.INVALID_PHONE	= "invalid_ph
ErrorCode.INVALID_DATE_OF_BIRTH	= "invalid_dat
ErrorCode.INVALID_ADDRESS	= "invalid_add

— Password —

ErrorCode.PASSWORD_TOO_WEAK	= "password_to
ErrorCode.PASSWORD_MISMATCH	= "password_mi
ErrorCode.PASSWORD_SAME_AS_OLD	= "password_sa

— File Upload —


```

ErrorCode.INVALID_IMAGE_SIZE           = "invalid_image_size"
ErrorCode.INVALID_IMAGE_TYPE           = "invalid_image_type"

# — Cart —
ErrorCode.CART_EMPTY                   = "cart_empty"
ErrorCode.CART_QUANTITY_EXCEEDS_STOCK = "cart_quantity_exceeds_stock"

# — Coupons —
ErrorCode.COUPON_INVALID               = "coupon_invalid"
ErrorCode.COUPON_EXPIRED               = "coupon_expired"
ErrorCode.COUPON_ALREADY_USED          = "coupon_already_used"
ErrorCode.COUPON_LIMIT_REACHED         = "coupon_limit_reached"
ErrorCode.COUPON_NOT_APPLIED           = "coupon_not_applied"

# — Orders —
ErrorCode.PRODUCT_UNAVAILABLE           = "product_unavailable"
ErrorCode.INSUFFICIENT_STOCK            = "insufficient_stock"
ErrorCode.PAYMENT_METHOD_UNAVAILABLE   = "payment_method_unavailable"
ErrorCode.ORDER_NOT_CANCELABLE         = "order_not_cancelable"

# — Logistics —
ErrorCode.ORDER_NOT_CONFIRMED           = "order_not_confirmed"
ErrorCode.SHIPMENT_ALREADY_EXISTS       = "shipment_already_exists"
ErrorCode.TRACKING_NUMBER_EXISTS        = "tracking_number_exists"
ErrorCode.INVALID_SHIPMENT_TRANSITION   = "invalid_shipment_transition"
ErrorCode.SHIPMENT_NOT_FOUND            = "shipment_not_found"
ErrorCode.COURIER_API_TIMEOUT           = "courier_api_timeout"
ErrorCode.ORDER_HAS_NO_SHIPMENT         = "order_has_no_shipment"

# — Logistics / Settlement —
ErrorCode.SETTLEMENT_ALREADY_EXISTS     = "settlement_already_exists"
ErrorCode.SETTLEMENT_ALREADY_RECONCILED = "settlement_already_reconciled"
ErrorCode.SHIPMENT_NOT_DELIVERED        = "shipment_not_delivered"
ErrorCode.SHIPMENT_ALREADY_SETTLED      = "shipment_already_settled"

# — Logistics / courier —
ErrorCode.COURIER_API_NOT_CONFIGURED    = "courier_api_not_configured"
ErrorCode.COURIER_API_ERROR             = "courier_api_error"

# — Reviews —

```

```

# Owner: apps.reviews
ErrorCode.ORDER_NOT_DELIVERED      = "order_not_deliver
ErrorCode.REVIEW_ALREADY_EXISTS     = "review_already_ex
ErrorCode.CANNOT_VOTE_OWN_REVIEW   = "cannot_vote_own_r
ErrorCode.REVIEW_NOT_APPROVED       = "review_not_approv

# — Returns —————
# Owner: apps.returns
ErrorCode.RETURN_WINDOW_EXPIRED     = "return_window_exp
ErrorCode.RETURN_ALREADY_EXISTS     = "return_already_ex
ErrorCode.PHOTOS_REQUIRED           = "photos_required"
ErrorCode.RETURN_NOT_APPROVED       = "return_not_approv
ErrorCode.REFUND_EXCEEDS_ORIGINAL   = "refund_exceeds_o

# — Payments —————
# Owner: apps.payments
ErrorCode.ORDER_NOT_PAYABLE         = "order_not
ErrorCode.INVALID_GATEWAY           = "invalid_g
ErrorCode.PAYMENT_ALREADY_SUCCESS   = "payment_a
ErrorCode.PAYMENT_GATEWAY_TIMEOUT   = "payment_g
ErrorCode.PAYMENT_GATEWAY_ERROR     = "payment_g
ErrorCode.PAYMENT_GATEWAY_NOT_CONFIGURED = "payment_g

# — Wishlist —————
# Owner: apps.wishlist
ErrorCode.WISHLIST_ITEM_ALREADY_EXISTS = "wishlist_it
ErrorCode.WISHLIST_ITEM_NOT_FOUND      = "wishlist_it

# — Top-level Category Codes (go into errors.code no
# These describe WHAT KIND of error occurred, not the
ErrorCode.VALIDATION_ERROR           = "validation_
ErrorCode.AUTHENTICATION_ERROR       = "authentica
ErrorCode.PERMISSION_ERROR           = "permission_
ErrorCode.NOT_FOUND                  = "not_found"
ErrorCode.RATE_LIMIT_EXCEEDED        = "rate_limit_
ErrorCode.SERVER_ERROR               = "server_erro
ErrorCode.METHOD_NOT_ALLOWED        = "method_not_
ErrorCode.CONFLICT_ERROR             = "conflict_er

```

When adding a new code:

1. Add constant to ErrorCode in apps/core/error_codes
2. Add a comment indicating which app/domain owns it
3. Never delete – deprecate with a comment instead
4. Inform the frontend team – these are a public cont

11. Two-Level Cache System

File: apps/core/cache.py

Singleton: from apps.core.cache import two_level_cache

Architecture

Request

|

v

L1: LocMemCache (caches["local"])
per-process, ultra-fast, 60s TTL, max 500 entries
miss?

|

v

L2: Redis (caches["default"])
shared across all workers, 300s TTL
miss?

|

v

Database (via builder_fn passed to get_or_set)

Key Concepts

Cache Versioning:

All keys prefixed with v{CACHE_VERSION}:

Currently CACHE_VERSION = 1

To bust entire cache on deploy: increment CACHE_V

Sentinel Encoding:

All values stored as {"__v": value}

Distinguishes genuine cache miss (backend returns
a cached falsy value (None, False, [], 0).

Without this, cached None looks like a miss → unn

Stampede Protection:

`get_or_set()` uses `cache.add()` (atomic SET NX) as
Only ONE worker rebuilds cache on cold miss.
All other concurrent workers wait up to `_lock_max`
If Redis unavailable: `_acquire_lock` fails open –
without stampede protection rather than crashing.

L1 Deep Copy:

`L1 get()` returns `copy.deepcopy(value)`.
Without this, callers mutating returned object would
shared in-process cache for all subsequent requests.

Fault Tolerance:

L1 and L2 failures are caught, logged as WARNING,
Cache failures degrade gracefully – they never crash.

Public API

```
from apps.core.cache import two_level_cache
```

```
# — Cache-aside (recommended pattern) —————
data, source = two_level_cache.get_or_set(
    "products:list:page1",
    lambda: MySerializer(MyModel.objects.all()), many=1,
    lock=True,                      # default True – set False in tests
    l1_timeout=60,                  # optional override of instance l1_timeout
    l2_timeout=300,                 # optional override of instance l2_timeout
)
```

```
# source: "l1_memory" | "l2_redis" | "database"
```

```
# — Manual get —————
data, source = two_level_cache.get("products:list:page1")
if source == "miss":
    data = build_data()
    two_level_cache.set("products:list:page1", data)
```

```
# — Set —————
two_level_cache.set(
    "my_key",
```

```

        value,
        l1_timeout=30,          # optional
        l2_timeout=120,        # optional
    )

# — Delete (prefix-based, SCAN safe – never KEYS *)
two_level_cache.delete("products:")
# L2: SCAN for *v1:products:* pattern
# L1: full clear (cheap – rebuilds on next request)

```

Return Values

get() and get_or_set() both return (value, source):

source	Meaning
-----	-----
"l1_memory"	Served from in-process LocMemCache
"l2_redis"	Served from Redis; L1 backfilled au
"miss"	Not in either level – caller must r
"database"	Cache was cold, builder_fn() called

Pass source to build_pagination_meta() to expose cache response meta.source field.

Instance Defaults

```
two_level_cache = TwoLevelCache(l1_timeout=60, l2_timeout=120)
```

Override per-call using l1_timeout / l2_timeout keywords
Different timeouts for different data profiles:

```

    Category list (rarely changes):  l2_timeout=3600
    Search results (change often):   l2_timeout=60
    User-specific data:               scope key to use

```

Cache Key Conventions

```

products:list:{filter_hash}
products:detail:{slug}
categories:tree
brands:list

```

```
    cart:{user_id}
    orders:list:{user_id}:page:{page}
```

12. Pagination System

File: apps/core/paginations.py

Two functions used together in every paginated view.

```
### get_pagination_params(request, default_page_size=20,

    from apps.core.paginations import get_pagination_params

    params = get_pagination_params(request)
    # Returns PaginationParams(page, page_size, is_valid)
    # PaginationParams is a NamedTuple – access as params

    if not params.is_valid:
        return self.error_response(message="Invalid pagination")
```

Behavior:

- Never raises exceptions – returns is_valid=False on error
- Logs WARNING on invalid params (visible in apps.core.log)
- page is always >= 1
- page_size is always between 1 and max_page_size
- If page is valid but page_size is not: keeps valid page
- Query params: ?page=2&page_size=12

Default limits:

```
    default_page_size=20 – returned when page_size not in query
    max_page_size=48 – hard ceiling regardless of what is requested
```

Override per endpoint:

```
    params = get_pagination_params(request, default_page_size=10)
```

```
### build_pagination_meta(page, page_size, total, *, source_url)

    meta = build_pagination_meta(
```

```

        params.page,
        params.page_size,
        total_count,
        source=cache_source,      # "l1_memory" | "l2_redis"
        sort=sort_param,         # "newest" | "price_asc"
        elapsed_ms=12.4,         # optional timing
    )

```

Returns:

```

{
    "page":          1,
    "page_size":     20,
    "total":         540,
    "total_pages":   27,
    "showing_from":  1,
    "showing_to":    20,
    "has_next":      True,
    "has_previous":  False,
    # Optional – only included if passed as non-None:
    "source":        "l2_redis",
    "sort":          "newest",
    "elapsed_ms":    12.4,
}

```

clamp_page=True (default):

If page overshoots total_pages, it is clamped down to total_pages
 Prevents showing_from/showing_to from producing garbage

Complete Paginated View Pattern

```

import time
from apps.core.paginations import get_pagination_params
from apps.core.cache import two_level_cache

```

```

class ProductListAPIView(BaseAPIView):
    def get(self, request):
        params = get_pagination_params(request, default_params)
        if not params.is_valid:
            return self.error_response(message="Invalid parameters")

```

```

sort = request.query_params.get("sort", "new")
cache_key = f"products:list:{sort}:page:{params.page}"

t0 = time.monotonic()

data, source = two_level_cache.get_or_set(
    cache_key,
    lambda: self._build_product_data(params, sort)
)

elapsed_ms = (time.monotonic() - t0) * 1000

meta = build_pagination_meta(
    params.page,
    params.page_size,
    data["total"],
    source=source,
    sort=sort,
    elapsed_ms=round(elapsed_ms, 2),
)

return self.success_response(
    data=data["results"],
    message="Products retrieved successfully.",
    meta=meta,
)

```

13. Serializer Fields and Mixins

Files: apps/core/fields.py, apps/core/mixins.py

CustomDateTimeField

```

from apps.core.fields import CustomDateTimeField

class MySerializer(serializers.ModelSerializer):
    some_date = CustomDateTimeField()

```


Defaults:

```
format=None
input_formats=['iso-8601']
default_timezone=timezone.get_current_timezone()
read_only=True
```

TimestampFieldsMixin

```
from apps.core.mixins import TimestampFieldsMixin

class MySerializer(TimestampFieldsMixin, serializers.
    class Meta:
        model = MyModel
        fields = [..., "created_at", "updated_at"]
```

Automatically adds created_at and updated_at as CustomDateField.
ISO-8601, read-only, timezone-aware.

Use this mixin on EVERY serializer that exposes timestamp fields.
Do NOT redefine these fields manually.

14. Permission Classes

File: apps/core/permissions.py

Role source: apps.common.choices.role.Role

All classes inherit from rest_framework.permissions.BasePermission

All denials are logged at WARNING level.

```
from apps.core.permissions import (
    IsAdmin,
    IsAdminOrReadOnly,
    IsCustomer,
    IsVerified,
    IsNotAuthenticated,
    RoleBasedProfilePermission,
)
```

Permission Class	Rule
------------------	------

-----	-----
-------	-------

IsAdmin		user.role == Role.ADMIN
IsAdminOrReadOnly		GET/HEAD/OPTIONS: anyone
		POST/PUT/PATCH/DELETE: A
IsCustomer		user.role == Role.CUSTOMER
IsVerified		user.is_verified == True
IsNotAuthenticated		not user.is_authenticated
RoleBasedProfilePermission		Verified CUSTOMER or ADMIN

Usage in Views:

```
class ProductListAPIView(BaseAPIView):
    permission_classes = [IsAdminOrReadOnly]

class CartDetailAPIView(BaseAPIView):
    permission_classes = [IsAuthenticated, IsVerified]
```

NOTE: BaseAPIView defaults to permission_classes = [AllowAllMethodPermission]
Always override explicitly for protected endpoints.

15. Middleware

File: apps/core/middleware.py

Position in MIDDLEWARE list: 2nd (after CorsMiddleware)

RequestIDMiddleware

```
class RequestIDMiddleware:
    def __call__(self, request):
        request.id = str(uuid.uuid4()) # unique ID
        response = self.get_response(request)
        response["X-Request-ID"] = request.id # also in headers
        return response
```

Every request has request.id.

BaseAPIView.transform_payload() reads this and injects in payload
Exception handler reads getattr(request, "id", None) – see below

16. Authentication and Token Strategy

JWT Configuration (SimpleJWT)

```
SIMPLE_JWT = {  
    'ACCESS_TOKEN_LIFETIME':    timedelta(minutes=1),  
    'REFRESH_TOKEN_LIFETIME':   timedelta(days=1),  
    'ROTATE_REFRESH_TOKENS':    True,           # new refresh token  
    'BLACKLIST_AFTER_ROTATION': True,           # old refresh token  
    'AUTH_HEADER_TYPES':        ('Bearer',),  
}
```

Token Storage Strategy

Token	Location	Rationale
-----	-----	-----
Access Token	Response body data	Frontend stores NEVER local storage
Refresh Token	HttpOnly cookie	JavaScript cannot access Browser sends with request

Cookie Settings for Refresh Token

```
Name:      refresh_token  
HttpOnly:  true  
Secure:    true (production) / false (local dev)  
SameSite:  lax  
MaxAge:    7 days  
Path:      /api/accounts/
```

Cookie scoped to /api/accounts/ – NOT sent on production

CSRF Strategy

```
CSRF Token: Set as readable cookie  
Frontend:   Reads cookie, sends in X-CSRFToken header  
Pattern:    Double-submit CSRF protection
```

Auth Flow for Protected Endpoints

1. Frontend sends Authorization: Bearer <access_token
2. DRF JWTAuthentication validates the token
3. If expired → frontend uses refresh token (auto-sends
POST /api/accounts/token/refresh/ for a new access
4. Frontend retries original request with new access

17. Logging System

Logger Names – One Per App

```
import logging
logger = logging.getLogger("apps.accounts")    # in ac
logger = logging.getLogger("apps.products")    # in pr
logger = logging.getLogger("apps.orders")      # in or
logger = logging.getLogger("apps.cart")        # in ca
logger = logging.getLogger("apps.core")        # in co
```

Use the correct logger for the correct app.
Do NOT use the root logger – it has level WARNING, DE

Log Files

Logger	File
apps.accounts	logs/accounts.log
apps.products	logs/products.log
apps.orders	logs/orders.log
apps.cart	logs/cart.log
apps.core	logs/core.log
django, celery	logs/django.log

All loggers also write to console. Level is DEBUG for

Log Format

```
[2024-01-15 14:30:22] INFO apps.products ProductListA
[YYYY-MM-DD HH:MM:SS] LEVEL name message
```

Logging Policy

```
In services (raise exceptions, add context):
    logger.info("Order created: order_id=%s user_id=%s")
    logger.warning("Inventory low: variant_id=%s stock=%s")
```

```
In exception handler (automatic – do not duplicate):
    DomainError                                → WARNING, no exc_info
    InfrastructureError notify=True             → ERROR, exc_info=True
    InfrastructureError notify=False           → WARNING, no exc_info
    DRF 4xx                                     → WARNING, no exc_info
    DRF 5xx                                     → ERROR, exc_info=True
    Unhandled                                  → ERROR, exc_info=True
```

```
NEVER do this – logging in builder functions or serializers
    logger.error("...", exc_info=True) # in _non_field_errors
    # Logging belongs in the handler and service layer
```

SensitiveDataFilter

```
Applied to the file handler for django.log.
Masks sensitive fields before writing to disk.
File: apps/core/logging.py
Do not log passwords, tokens, or PII in any log call.
```

18. Settings Reference

File: config/settings/base.py (single settings file – no overrides)

Key Settings

Setting	Value
AUTH_USER_MODEL	"accounts.User"
TIME_ZONE	"UTC" (database timezone)
CELERY_TIMEZONE	"Asia/Karachi" (PKT for scheduler)
DEFAULT_AUTO_FIELD	"django.db.models.BigAutoField"

EXCEPTION_HANDLER		"apps.core.api.exceptions.
DEFAULT_SCHEMA_CLASS		"drf_spectacular.openapi.A
Redis port		6380 (NOT default 6379 – i
Access token lifetime		1 minute
Refresh token lifetime		1 day

Cache Backend Names (exact strings used in TwoLevelCa

 caches["default"] → Redis (L2, shared across worker
 caches["local"] → LocMemCache (L1, per-process)

Do not change these names without updating apps/core/

Installed Apps Load Order

 1. DJANGO_APPS (django.contrib.*)
 2. THIRD_PARTY_APPS (rest_framework, simplejwt, cor
 3. PROJECT_APPS (apps.core, apps.common, apps.a

Default Throttle Rates

 CustomAnonRateThrottle → 10000/hour
 CustomUserRateThrottle → 1000/hour
 Custom throttle classes in apps/core/throttles.py

Debug Toolbar

Enabled ONLY when DEBUG=True – middleware appended co
INTERNAL_IPS includes Docker internal IP detection lo

Timezone Critical Note

 TIME_ZONE = "UTC" for the database.
 CELERY_TIMEZONE = "Asia/Karachi" for scheduled tasks.
 When bounding date ranges in nightly analytics aggreg
 use timezone.localtime() to avoid midnight bleeding a
 Failure to do this produces daily snapshots that span

19. Critical Rules – Never Violate

These are enforced by architecture. Violating them causes hard to trace and inconsistencies that break the frontend

Response Shape Rules

1. Every view MUST inherit from BaseAPIView – never A
2. Never construct a raw Response() in a view.
Always use the helper methods: success_response(),
3. Never put data in the errors field or errors in th
4. meta must always contain request_id.
This is automatic via transform_payload(). Do not

Exception Rules

5. Raise DomainError and InfrastructureError in the s
Let custom_exception_handler convert them to respo
Do NOT catch and re-raise as DRF APIException – yo
6. Never pass exc.internal to any builder function.
It must only appear in log calls inside custom_exc
7. Never log inside builder functions (_non_fields_do
ONE log entry per exception. Logging belongs in cu
8. DomainError status_code MUST be 400 or 409 only.
The constructor raises ValueError if you pass anyt
9. InfrastructureError is always 503. You cannot chan
If you need a different 5xx, use DRF APIException

Error Code Rules

10. Never use string literals at raise sites.
Always import and use ErrorCode constants from ap

11. Never DELETE an ErrorCode – deprecate with a comment. The frontend switches on these values. Deleting b

Cache Rules

12. Never call KEYS * on Redis in any context.
Always use delete_pattern() which uses SCAN internally.
13. Never cache user-specific sensitive data in shared cache without scoping the cache key to the user ID.
14. Never convert Decimal monetary values to float due to precision.
Use decimal.Decimal and Django Sum() with DecimalField.

Inventory and Transaction Rules

15. Wrap checkout logic in transaction.atomic().
Use ProductVariant.objects.select_for_update() with lock=True.
Without this, concurrent checkouts WILL oversell.
16. Never mutate order status without writing an OrderStatusChange.
Use a service method that atomically updates both order and status.

Logging Rules

17. Use the correct logger for the correct app.
logger = logging.getLogger("apps.products") in products app
logger = logging.getLogger("apps.orders") in orders app
18. Never use the root logger – logging.getLogger() without app name.
Root logger level is WARNING. DEBUG/INFO messages are ignored.
19. Celery tasks that call external APIs MUST use explicit logging.
Never run synchronous external HTTP calls in the Celery task.

Timezone Rules

20. TIME_ZONE = "UTC" for the database.
CELERY_TIMEZONE = "Asia/Karachi" for scheduled tasks.
Always use timezone.localtime() when bounding data by time.

nightly analytics – never assume UTC midnight = P

20. How to Add a New Endpoint – Checklist

Follow this every time. Skip no step.

Step 1 – Models

- [] Model defined in apps/{app}/models.py
- [] Migration created and applied
- [] `__str__` defined on every model
- [] Correct `on_delete`: PROTECT for financial records,
- [] `BigAutoField` as default (set globally – no action

Step 2 – Error Codes

- [] Any new error codes added to apps/core/error_code
- [] Grouped with a comment indicating which app they
- [] Frontend team notified of new codes

Step 3 – Exceptions (if needed)

- [] Service raises `DomainError` for business rule viol
- [] Service raises `InfrastructureError` for external s
- [] `exc.internal` contains ONLY diagnostic data (never
- [] `exc.client_extra` contains ONLY data safe to expos

Step 4 – Service Layer

- [] Business logic in apps/{app}/services.py – never
- [] DB queries via selector/repository functions – ne
- [] `transaction.atomic()` wrapping any multi-step writ
- [] `select_for_update()` on any inventory/stock rows b

Step 5 – Serializer

- [] Serializer in apps/{app}/serializers.py
- [] Timestamp fields use `TimestampFieldsMixin` or Cust

```
[ ] No business logic in serializers – validation only
[ ] error_response(errors=serializer.errors) for validation
```

Step 6 – View

```
[ ] View inherits from BaseAPIView – never APIView or
[ ] permission_classes set explicitly (do NOT rely on
[ ] Paginated lists use get_pagination_params() + build
[ ] Cache via two_level_cache.get_or_set() for read-h
[ ] Correct response helper used: success_response /
[ ] Logger declared: logger = logging.getLogger("apps
[ ] Logger calls present for key actions: create, upd
```

Step 7 – URLs

```
[ ] URL added to apps/{app}/urls.py with app_name set
[ ] Named with descriptive name: name="order-detail"
[ ] Registered in config/urls.py under correct API pr
```

Step 8 – Testing (Postman)

```
[ ] Full envelope shape verified: success, message, d
[ ] request_id appears in meta
[ ] errors.code matches expected value for error scen
[ ] errors.non_fields.category is correct: domain / s
[ ] exc.internal does NOT appear in response body
[ ] Pagination: page, page_size, showing_from, showin
[ ] Cache: second request shows source="l2_redis" or
```

21. Appendix A – Response Examples by Scenario

200 – Paginated List

```
{
    "success": true,
    "message": "Products retrieved successfully.",
    "data": [
        {"id": 1, "name": "Chain Sprocket", "price":
```

```
],
"errors": null,
"meta": {
  "request_id": "abc123",
  "page": 1,
  "page_size": 12,
  "total": 54,
  "total_pages": 5,
  "showing_from": 1,
  "showing_to": 12,
  "has_next": true,
  "has_previous": false,
  "source": "l2_redis",
  "sort": "newest"
}
}
```

201 – Resource Created

```
{
  "success": true,
  "message": "Order placed successfully.",
  "data": {
    "order_id": "ORD-2024-00123",
    "total": "4500.00"
  },
  "errors": null,
  "meta": {"request_id": "abc123"}
}
```

400 – Field Validation Failure

```
{
  "success": false,
  "message": "Registration failed.",
  "data": null,
  "errors": {
    "code": "validation_error",
    "fields": {
      "email": {
```

```
        "message": "Enter a valid email address",
        "code": "invalid_email_format"
    },
    "non_fields": null
},
"meta": {"request_id": "abc123"}
}
```

400 – Domain Rejection

```
{
  "success": false,
  "message": "Your cart is empty.",
  "data": null,
  "errors": {
    "code": "validation_error",
    "fields": null,
    "non_fields": {
      "category": "domain",
      "message": "Your cart is empty. Add item to cart.",
      "code": "cart_empty",
      "extra": null
    }
  },
  "meta": {"request_id": "abc123"}
}
```

401 – Authentication Required

```
{
  "success": false,
  "message": "Authentication required.",
  "data": null,
  "errors": {
    "code": "authentication_error",
    "fields": null,
    "non_fields": {
      "category": "validation",
      "message": "Authentication credentials were not provided.",
      "code": "authentication_credentials_not_provided",
      "extra": null
    }
  },
  "meta": {"request_id": "abc123"}
}
```

```
        "code":      "authentication_error",
        "extra":     null
    },
    "meta": {"request_id": "abc123"}
}
```

409 – State Conflict

```
{
  "success": false,
  "message": "This order has already been cancelled",
  "data": null,
  "errors": {
    "code": "conflict_error",
    "fields": null,
    "non_fields": {
      "category": "domain",
      "message": "This order has already been",
      "code": "order_already_cancelled",
      "extra": null
    }
  },
  "meta": {"request_id": "abc123"}
}
```

503 – Infrastructure Failure

```
{
  "success": false,
  "message": "The service is temporarily unavailable",
  "data": null,
  "errors": {
    "code": "server_error",
    "fields": null,
    "non_fields": {
      "category": "system",
      "message": "Payment gateway is temporarily",
      "code": "payment_gateway_timeout",
      "extra": {"retry_after": 30}
    }
  }
}
```

```

        }
    },
    "meta": {"request_id": "abc123"}
}

```

22. Appendix B – Import Reference

Core infrastructure – use these, never reinvent

```

from apps.core.api.views          import BaseAPIView
from apps.core.api.exceptions import (
    _format_drf_errors,
    _non_fields_domain,
    _status_to_error_code,
    register_monitor,
)
from apps.core.exceptions          import DomainError, Inf
from apps.core.error_codes         import ErrorCode
from apps.core.cache               import two_level_cache
from apps.core.paginations         import get_pagination_p
from apps.core.fields              import CustomDateTimeFi
from apps.core.mixins              import TimestampFieldsM
from apps.core.permissions         import (
    IsAdmin,
    IsAdminOrReadOnly,
    IsCustomer,
    IsVerified,
    IsNotAuthenticated,
    RoleBasedProfilePermission,
)

```

Document version: 1.0

Last updated: Based on actual source files read in full.
 Do not modify this document without reading the actual so
 Do not guess at implementation details – ask for the file
 ``

